Tugas Kecil 2: Voxelization Objek 3D menggunakan Octree

IF2211 Strategi Algoritma
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Muhammad Iqbal Raihan (13524011)

Repository: https://github.com/Gixgine-budi/Tucil2_13524011

28 Maret 2026

Daftar Isi

1 Latar Belakang **3**

2 Dasar Teori **3**
2.1 *Divide and conquer* . . . 3
2.2 Deteksi *collision* antara *voxel* dan muka segitiga . . . 3
2.2.1 Tes *AABB* pada sumbu kartesian . . . 4
2.2.2 Straddle terhadap bidang segitiga . . . 4
2.2.3 *Separating axis theorem* (SAT) . . . 4
2.3 Konkurensi dan paralelisasi . . . 4

3 Implementasi Algoritma **5**
3.1 *Divide and conquer* . . . 5
3.2 Representasi data . . . 5
3.3 Alur utama . . . 6
3.4 *Octree* subdivision and exploration . . . 6
3.5 Rutin *collision* (`checkCollision`) . . . 6
3.6 Ekspor OBJ dan statistik . . . 6
3.7 I/O dan pelaporan CLI . . . 6

4 Analisis Kompleksitas **6**
4.1 Kompleksitas waktu . . . 6
4.2 Kompleksitas ruang . . . 7

5 Pengujian **7**
5.1 Kasus uji . . . 7
5.1.1 Kasus uji 1 . . . 7
5.1.2 Kasus uji 2 . . . 9
5.1.3 Kasus uji 3 . . . 10
5.1.4 Kasus uji 4 . . . 12
5.1.5 Kasus uji 5 . . . 14
5.2 Ringkasan hasil pengujian . . . 17

6 Kesimpulan **17**

A Checklist penilaian **17**

1 Latar Belakang

Dalam IF2211 Strategi Algoritma, salah satu pendekatan dasar untuk masalah besar adalah *divide and conquer*: memecah masalah menjadi submasalah sejenis yang lebih kecil, menyelesaikannya, lalu menggabungkan hasil bila perlu. Tugas Kecil 2 semester II 2025/2026 meminta software yang mengonversi model 3D dalam Wavefront OBJ menjadi representasi berbasis kubus kecil (*voxelization*), mirip estetika blok pada *Minecraft*. Spesifikasi mensyaratkan struktur data *octree* dan algoritma *divide and conquer*.

Dalam konteks ini, *voxelization* memetakan objek yang direpresentasikan dengan *mesh* segitiga ke kumpulan *voxel* berukuran seragam pada *leaf depth* maksimum; hanya permukaan yang diproses: *leaf* yang berpotongan dengan geometri segitiga dipertahankan sebagai *voxel*, ruang kosong tidak dibelah lebih jauh. *Octree* memodelkan *spatial partitioning*: setiap *node* adalah sel kubus; *internal nodes* punya delapan anak (*octants*); subdivisi 3D bersifat rekursif.

Program `ttov` ditulis dalam Go. Program membaca OBJ minimal (`v`, `f`), membangun *axis-aligned bounding cube* untuk seluruh *vertices*, lalu mengeksplorasi *octree* sampai *depth limit* dari pengguna. Hanya cabang yang masih memotong segitiga yang terus di-*subdivide*; cabang kosong di-*prune* lebih awal. Output berupa satu file OBJ yang berisi *quad mesh* (enam *quads* per *voxel*) dengan *vertex deduplication*, ditulis ke `output4.obj` di *working directory*. Tugas dikerjakan secara individu tanpa pembagian tugas: *design*, *implementation*, *testing*, dan laporan menjadi tanggung jawab penulis.

2 Dasar Teori

2.1 *Divide and conquer*

Divide and conquer adalah strategi algoritmik di mana suatu masalah dibagi menjadi satu atau beberapa kasus sejenis yang lebih kecil, setiap kasus diselesaikan, lalu hasilnya digabungkan. Kompleksitas sering membaik bila kasus saling independen dan ukuran masalah turun secara signifikan pada tiap pembagian; pada banyak masalah geometri, pembagian yang cerdas juga memungkinkan membuang seluruh cabang yang tidak relevan tanpa menghitung solusi parsialnya.

Pada *voxelization* berbasis ruang 3D, masalah induk dapat dipandang sebagai sel kubus awal yang mencakup *mesh*. Langkah *divide*: jika sel berpotongan dengan segitiga dan belum mencapai kedalaman batas, sel dipecah menjadi delapan subruang yang sama besar. Tiap subruang membawa subset segitiga yang mungkin memotong subruang tersebut (*culling*), sehingga submasalah lebih kecil baik secara geometri maupun jumlah segitiga. Jika sel tidak berpotongan dengan segitiga sama sekali, cabang di bawahnya tidak perlu dibangun (*pruning*). Langkah *conquer*: pada kedalaman maksimum, setiap sel yang masih berpotongan dengan segitiga ditetapkan sebagai *voxel* permukaan. Penggabungan hasil pada akhirnya berupa pengumpulan daun yang terisi.

Struktur data yang memfasilitasi pembagian delapan arah ini adalah *octree*: setiap *node* menyimpan sel kubus *axis-aligned*; *internal node* memiliki hingga delapan anak (*octants*); tiap level membagi panjang rusuk menjadi setengah sehingga volume tiap anak seperdelapan dari induk. *Octree* bukan algoritma tersendiri melainkan kerangka spasial tempat langkah *divide* dan *prune* dijalankan secara terstruktur.

2.2 Deteksi *collision* antara *voxel* dan muka segitiga

Setiap keputusan subdivisi dan penandaan *leaf* bergantung pada kondisi apakah kubus sel (*voxel*, model sebagai *AABB*) berpotongan dengan muka segitiga. Pada implementasi, jawaban ini dihitung sebagai rangkaian tes bertingkat, dimulai dengan pengecekan AABB, diikuti dengan pengecekan berbasis *separating axis theorem* (SAT).

2.2.1 Tes *AABB* pada sumbu kartesian

Axis-aligned bounding box (*AABB*) kubus sel sejajar dengan sumbu x, y, dan z. Segitiga juga dibungkus *AABB* dari ketiga titik sudutnya. Dua *AABB* tidak mungkin berpotongan jika pada salah satu sumbu interval proyeksi keduanya terpisah.

2.2.2 Straddle terhadap bidang segitiga

Segitiga membentuk bidang dengan normal $\mathbf{n} = (\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a})$ untuk sudut $\mathbf{a}, \mathbf{b}, \mathbf{c}$. Untuk tiap sudut kubus dihitung nilai yang setara dengan jarak bertanda ke bidang melalui $\mathbf{a}$. Jika semua nilai bertanda tidak berubah tanda (semua tidak negatif atau semua tidak positif tanpa melintasi nol), seluruh kubus terletak di satu sisi bidang sehingga tidak memotong segitiga. Jika ada perubahan tanda, kubus memotong bidang tak-hingga yang dibentuk oleh segitiga, sehingga pasangan tetap menjadi kandidat dan dilanjutkan ke tahap SAT.

2.2.3 *Separating axis theorem* (SAT)

Untuk dua himpunan konveks di $\mathbb{R}^3$, keduanya terpisah (tanpa overlap volume interior) jika terdapat sumbu sedemikian sehingga proyeksi ortogonal keduanya ke sumbu itu berupa dua interval yang tidak overlap. Jika tidak ditemukan sumbu pemisah dalam himpunan sumbu uji yang memadai, untuk pasangan segitiga versus *AABB* disimpulkan ada overlap. Teorema ini disebut *separating axis theorem* (SAT).

Himpunan sumbu uji untuk segitiga dan *AABB* mencakup arah yang relevan dengan normal bidang (telah tercakup oleh tes straddle), vektor rusuk yang sejajar sumbu kubus, dan sumbu dari *cross product* $\mathbf{e}_\triangle \times \mathbf{d}_{\text{kubus}}$ dengan $\mathbf{e}_\triangle$ vektor sisi segitiga dan $\mathbf{d}_{\text{kubus}}$ vektor rusuk kubus. Pada tiap sumbu yang tidak *degenerate* (bukan vektor nol), titik sudut kubus dan titik sudut segitiga diproyeksikan lewat *dot product*; diperoleh interval $[\min, \max]$ untuk masing-masing bentuk. Jika $\max_{\text{kubus}} \leq \min_\triangle$ atau $\min_{\text{kubus}} \geq \max_\triangle$, sumbu tersebut memisahkan keduanya. Pada implementasi ini, kontak *touching-only* (tepat pada batas interval) tidak dianggap sebagai tabrakan. Setelah semua sumbu diuji tanpa pemisah, disimpulkan ada perpotongan. Rangkaian tes keseluruhan adalah *AABB*, straddle bidang, lalu loop SAT pada sumbu hasil *cross product* sisi segitiga dengan rusuk kubus.

2.3 Konkurensi dan paralelisasi

Konkurensi berarti beberapa alur eksekusi (*threads*, *goroutine*, atau proses) berjalan bersamaan dalam satu periode waktu; pada CPU multi-*core*, bagian pekerjaan yang independen dapat dijadwalkan paralel sehingga wall-clock time berkurang. Syarat utamanya adalah minimnya ketergantungan data serta perlindungan penulisan bersama (misalnya dengan `mutex`), karena *data race* merusak konsistensi pohon atau struktur antrean.

Pada eksplorasi *octree*, submasalah yang bersesuaian dengan *octant* berbeda di level atas saling independen asalkan masing-masing menerima salinan atau referensi aman ke data segitiga yang sudah disaring. Oleh karena itu delapan cabang pertama dapat diluncurkan sebagai tugas konkuren terpisah. Namun jumlah tugas yang dapat dijadwalkan efisien terbatas; jika *goroutine* yang aktif jauh melebihi jumlah *logical core*, biaya *scheduling* dan *stack* meningkat. Pola mitigasi yang dipakai program adalah ketika jumlah *goroutine* aktif melampaui kira-kira dua kali `GOMAXPROCS`, pembangunan *subtree* dilanjutkan secara iteratif dengan *breadth-first queue*, yang tetap benar secara semantik tetapi membatasi ledakan konkurensi.

3 Implementasi Algoritma

3.1 *Divide and conquer*

Secara konseptual, pipeline berikut menjelaskan apa yang dilakukan tiap tahap sebelum masuk ke detail tipe data dan fungsi, dari *mesh* segitiga yang valid sampai *file* OBJ berisi *voxel*. Urutan ini mengikuti pola *divide and conquer* pada ruang 3D, yaitu menyaring geometri yang relevan lalu memecah sel hanya bila masih ada perpotongan dengan segitiga.

1. Memvalidasi dan memuat *mesh*.

   Program membaca baris `v` dan `f`, menolak format yang tidak sesuai, lalu menyusun *vertex list* dan *triangle list* sebagai satu-satunya sumber geometri untuk tahap berikutnya.

2. Menentukan kubus pembatas *axis-aligned*.

   Dibangun kubus terkecil sejajar sumbu yang mencakup semua *vertices*. Panjang rusuk diambil sebagai max dari rentang x, y, dan z agar domain subdivisi berbentuk kubus, bukan balok sembarang.

3. *Subdivide* dan *cull* segitiga untuk tiap sel.

   Kubus besar dipecah menjadi delapan *octants*, setiap sel diuji interseksinya dengan segitiga, dan hanya subset segitiga yang benar-benar memotong *child cell* yang dibawa ke level berikutnya hingga *depth limit*. Sel yang kosong tidak dibelah lagi sehingga biaya komputasi tetap terkendali.

4. Menandai *surface voxel* pada kedalaman maksimum.

   Pada batas kedalaman, setiap sel yang masih berpotongan dengan segitiga ditandai sebagai *voxel* permukaan.

5. Mengekspor OBJ dengan *vertex deduplication*.

   Kumpulan kubus diubah menjadi baris `v` dan `f` pada format OBJ (*quads*), kemudian *vertices* pada posisi yang sama digabung agar ukuran berkas tidak terlalu besar akibat titik duplikat di sisi antar-*voxel* yang bersebelahan.

3.2 Representasi data

Tipe utama: `vertex` untuk titik 3D; `face` untuk segitiga berindeks ke `objFile.vertices`; `voxel` sebagai kubus (`base`, `size`); `octree` sebagai *tree node* dengan pointer `voxel`, `depth`, flag `fill`, delapan anak. Output voxel: `objFile4` berisi `face4` (*quads*).

Listing 1: Core types (struct.go)

```go
type objFile struct {
    vertices []vertex
    faces    []face
}

type octree struct {
    size     float64
    root     *voxel
    depth    int
    fill     bool
    children [8]*octree
}
```

3.3 Alur utama

Algorithm 1 Octree-based voxelization (outline)

Require: Valid triangle OBJ file, non-negative integer L (*subdivision depth limit*)
Ensure: File `output4.obj` with *surface voxels* as axis-aligned cubes

```
Parse and validate v and f lines
Compute bounding cube covering all vertices
Build octree: eight top-level octants in parallel, then recursive/iterative per node
At depth L, mark leaves intersecting triangles as voxels
Emit filled leaves as quads, dedupe vertices, write OBJ
```

3.4 *Octree* subdivision and exploration

`voxel.subdivide` memecah sel menjadi delapan kubus dengan *edge length* setengahnya. `exploreObj` membuat akar dan menjalankan eksplorasi untuk delapan anak *world cube*. Rutin paralel menyaring segitiga yang *collide* dengan sel saat ini. Tanpa segitiga, *node* kosong dan tanpa anak. Pada *depth limit*, `fill` ditentukan berdasarkan apakah masih ada interseksi dengan segitiga (*surface voxel*). Sebelum mencapai *depth limit*, jika masih ada segitiga, delapan anak dibangun dengan subset yang sudah di-*cull*.

3.5 Rutin *collision* (`checkCollision`)

`checkCollision` mengimplementasikan tes interseksi antara *voxel* dan segitiga: AABB, bidang, lalu SAT pada *edges*. Jika ada *separating axis*, mengembalikan `false` (no intersection).

3.6 Ekspor OBJ dan statistik

`octToObj` melakukan *BFS traversal* untuk menghitung *nodes* per *depth*, perkiraan *nodes not explored* pada level berikutnya ketika cabang kosong, dan jumlah *leaf voxels* yang ditulis. `compactObj4` menggabungkan *vertices* identik.

3.7 I/O dan pelaporan CLI

Input interaktif dari *standard input*: path file OBJ, lalu *subdivision level* (berapa kali kubus dipecah menjadi delapan; contoh pada program: level 3 ⇒ hingga 8^3 *finest cells* sepanjang satu jalur penuh). Program mencetak waktu untuk OBJ *scan*, voxelization, konversi ke struktur OBJ, *compaction*, dan *write*, plus waktu total. Juga jumlah *voxels*, *vertices*, *faces* setelah *compaction*, tabel *nodes per depth*, tabel *nodes not explored per depth*. Output file: `output4.obj` di *working directory*.

Validasi OBJ: wajib `v` dengan tiga koordinat dan `f` dengan tiga indeks segitiga (1-based di file); baris yang diawali `#` diabaikan; tipe baris yang tidak dikenal ⇒ error.

4 Analisis Kompleksitas

4.1 Kompleksitas waktu

Misalkan F jumlah segitiga input dan L *depth limit*. Per *node*, pada kasus terburuk semua segitiga yang dibawa ikut diuji terhadap sel; biaya per pasangan bersifat konstan (ditentukan oleh rutin interseksi geometri). *Tree* pada kasus terlebar memiliki $\sim 8^L$ *leaves*, namun *pruning* sel kosong mengurangi *nodes* yang dibangun untuk *mesh* yang tidak memenuhi seluruh ruang. Tanpa *pruning*, waktu kasus terburuk bersifat eksponensial terhadap L dan bergantung secara linear terhadap F. Pada praktiknya jumlah *nodes* jauh lebih kecil, tercermin pada kolom *nodes not explored* di keluaran program.

4.2 Kompleksitas ruang

Penyimpanan *tree* sebanding dengan jumlah *nodes* yang dialokasi; mode iteratif menyimpan *frontier queue*. Output menyimpan $O(V)$ *vertices* unik dan $O(V)$ *quads* untuk V *leaf voxels* sebelum *compaction*, berkurang setelah *deduplication* antarkubus yang berbagi sisi.

5 Pengujian

Spesifikasi mensyaratkan screenshot input dan output program untuk setidaknya lima contoh berbeda. Lingkungan uji: `go 1.26.1`, AMD Ryzen 7 7435HS, Windows 11.

5.1 Kasus uji

5.1.1 Kasus uji 1

- Input

  OBJ path: `test/cow.obj`, *subdivision level*: 8.

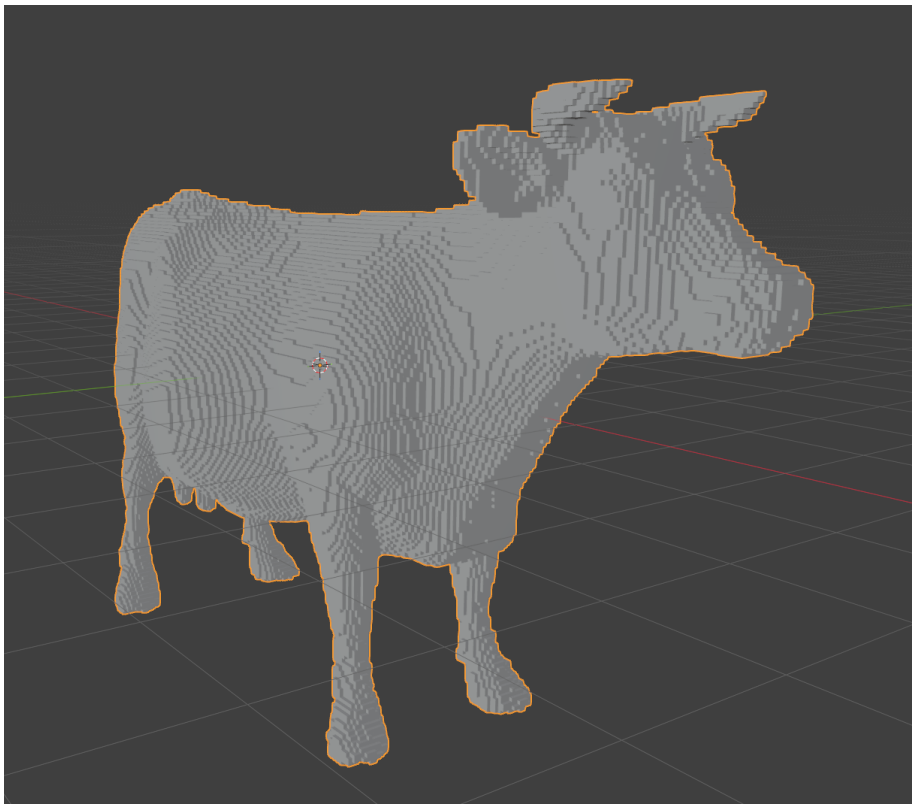
- Expected behavior

  Program menulis `output4.obj` dan mencetak jumlah *voxel*, *vertex*, dan *face*, statistik *node* per *depth*, jumlah *nodes not explored* per *depth*, serta *timings*.

- Hasil

```
$ .\build.bat run
Building ttov...
Enter file path: test/cow.obj

Subdivision level is the number of times the cube is subdivided into 8 smaller cubes.
For example, a subdivision level of 3 means the cube is subdivided into 8^3 = 512 smaller
Enter subdivision level: 8


============================================================
Scanning .obj file completed
Time taken to scan obj file: 22.3386ms

Octree generation completed
Time taken to voxelize obj file: 169.6209ms

Octree to OBJ data type conversion completed
Time taken to convert octree to OBJ data type: 124.1921ms

OBJ data has been succesfully compacted
Time taken to compact OBJ data: 343.632ms

Number of voxels: 91951
Number of vertices: 306960
Number of faces: 551706

Number of nodes at each depth:
1       : 1
```

```
2        : 8
3        : 32
4        : 152
5        : 696
6        : 2728
7        : 10928
8        : 45248
9        : 182424

Number of nodes not explored at each depth:
1        : 0
2        : 0
3        : 32
4        : 104
5        : 520
6        : 2840
7        : 10896
8        : 42176
9        : 179560


OBJ file writing completed
Time taken to write obj file: 183.0416ms

Total time taken: 842.8252ms
```



Gambar 1: Test case 1

5.1.2 Kasus uji 2

- Input

  OBJ path: `test/pumpkin.obj`, *subdivision level*: 9.

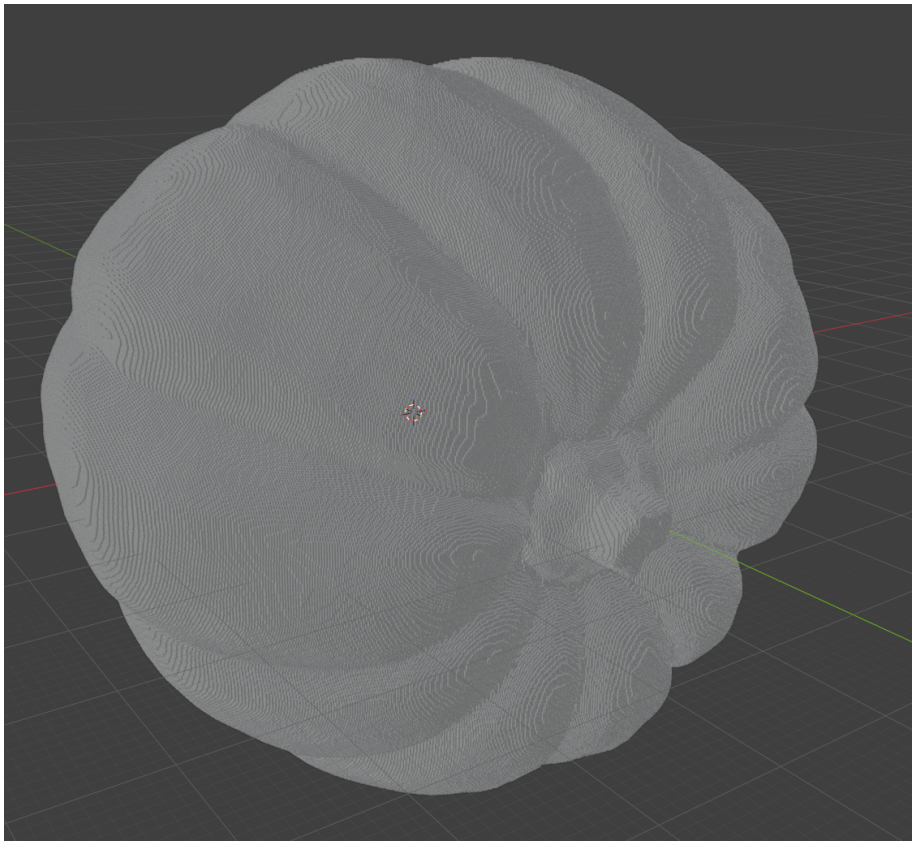
- Expected behavior

  Program menulis `output4.obj` dan mencetak jumlah *voxel*, *vertex*, dan *face*, statistik *node* per *depth*, jumlah *nodes not explored* per *depth*, serta *timings*.

- Hasil

```
$ .\build.bat run
Building ttov...
Enter file path: test/pumpkin.obj

Subdivision level is the number of times the cube is subdivided into 8 smaller cubes.
For example, a subdivision level of 3 means the cube is subdivided into 8^3 = 512 smaller
Enter subdivision level: 9


============================================================
Scanning .obj file completed
Time taken to scan obj file: 40.6941ms

Octree generation completed
Time taken to voxelize obj file: 832.4522ms

Octree to OBJ data type conversion completed
Time taken to convert octree to OBJ data type: 1.7177806s

OBJ data has been succesfully compacted
Time taken to compact OBJ data: 7.8247703s

Number of voxels: 1196633
Number of vertices: 3960457
Number of faces: 7179798

Number of nodes at each depth:
1       : 1
2       : 8
3       : 64
4       : 448
5       : 2024
6       : 8864
7       : 36824
8       : 148864
9       : 596992
10      : 2391672

Number of nodes not explored at each depth:
1       : 0
```

```
2        : 0
3        : 0
4        : 64
5        : 1560
6        : 7328
7        : 34088
8        : 145728
9        : 593920
10       : 2384264


OBJ file writing completed
Time taken to write obj file: 3.5628761s

Total time taken: 13.9785733s
```



Gambar 2: Test case 2

5.1.3 Kasus uji 3

- Input

  OBJ path: `test/teapot.obj`, *subdivision level*: 8.

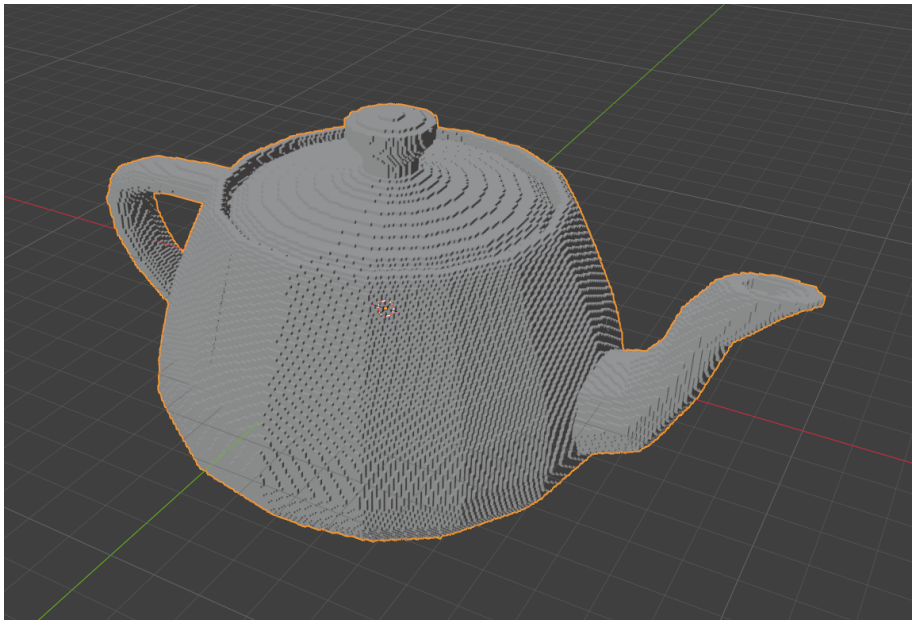
- Expected behavior

  Program menulis `output4.obj` dan mencetak jumlah *voxel*, *vertex*, dan *face*, statistik *node* per *depth*, jumlah *nodes not explored* per *depth*, serta *timings*.

- Hasil

```
$ .\build.bat run
Building ttov...
Enter file path: test/teapot.obj

Subdivision level is the number of times the cube is subdivided into 8 smaller cubes.
For example, a subdivision level of 3 means the cube is subdivided into 8^3 = 512 smaller
Enter subdivision level: 8


==============================================================
Scanning .obj file completed
Time taken to scan obj file: 32.6059ms

Octree generation completed
Time taken to voxelize obj file: 102.9254ms

Octree to OBJ data type conversion completed
Time taken to convert octree to OBJ data type: 145.3819ms

OBJ data has been succesfully compacted
Time taken to compact OBJ data: 383.2267ms

Number of voxels: 118263
Number of vertices: 329286
Number of faces: 709578

Number of nodes at each depth:
1       : 1
2       : 8
3       : 32
4       : 160
5       : 784
6       : 3440
7       : 14272
8       : 58400
9       : 236152

Number of nodes not explored at each depth:
1       : 0
2       : 0
3       : 32
4       : 96
5       : 496
6       : 2832
7       : 13248
8       : 55776
9       : 231048


OBJ file writing completed
```

```
Time taken to write obj file: 252.1601ms

Total time taken: 916.3ms
```



Gambar 3: Test case 3

5.1.4 Kasus uji 4

- Input

  OBJ path: `test/line.obj`, *subdivision level*: 5.

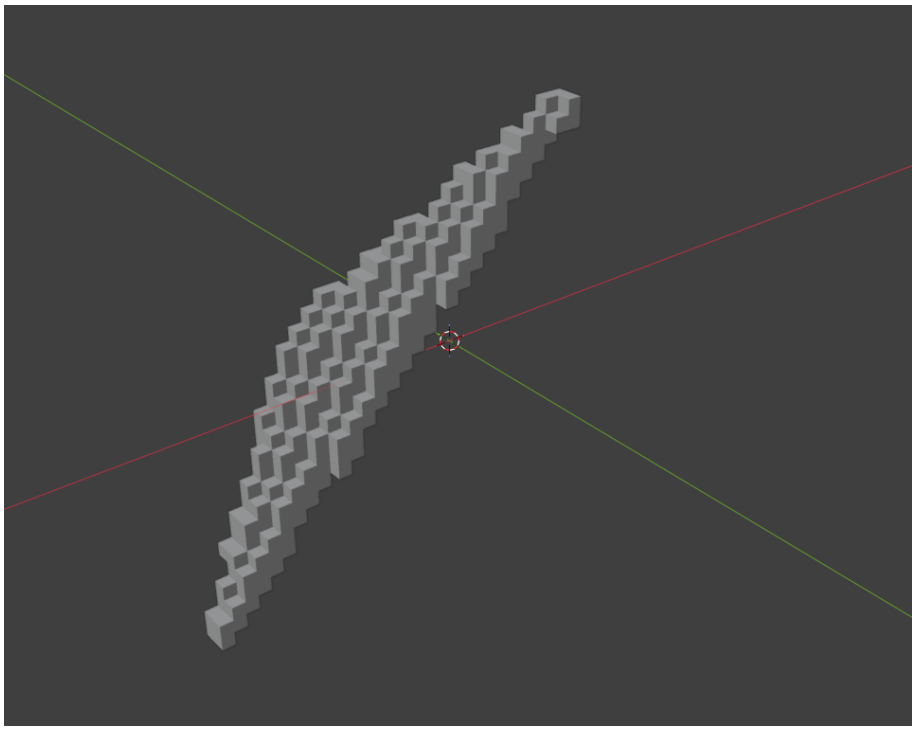
- Expected behavior

  Program menulis `output4.obj` dan mencetak jumlah *voxel*, *vertex*, dan *face*, statistik *node* per *depth*, jumlah *nodes not explored* per *depth*, serta *timings*.

- Hasil

```
$ .\build.bat run
Building ttov...
Enter file path: test/line.obj

Subdivision level is the number of times the cube is subdivided into 8 smaller cubes.
For example, a subdivision level of 3 means the cube is subdivided into 8^3 = 512 smaller
Enter subdivision level: 5


============================================================
Scanning .obj file completed
Time taken to scan obj file: 1.0979ms

Octree generation completed
Time taken to voxelize obj file: 2.7027ms
```

```
Octree to OBJ data type conversion completed
Time taken to convert octree to OBJ data type: 552.5µs

OBJ data has been succesfully compacted
Time taken to compact OBJ data: 1.0945ms

Number of voxels: 288
Number of vertices: 868
Number of faces: 1728

Number of nodes at each depth:
1       : 1
2       : 8
3       : 24
4       : 56
5       : 216
6       : 672

Number of nodes not explored at each depth:
1       : 0
2       : 0
3       : 40
4       : 136
5       : 232
6       : 1056


OBJ file writing completed
Time taken to write obj file: 7.3778ms

Total time taken: 12.8254ms
```



Gambar 4: Test case 4

5.1.5 Kasus uji 5

- Input

  OBJ path: `test/chen.obj`, *subdivision level*: 10.

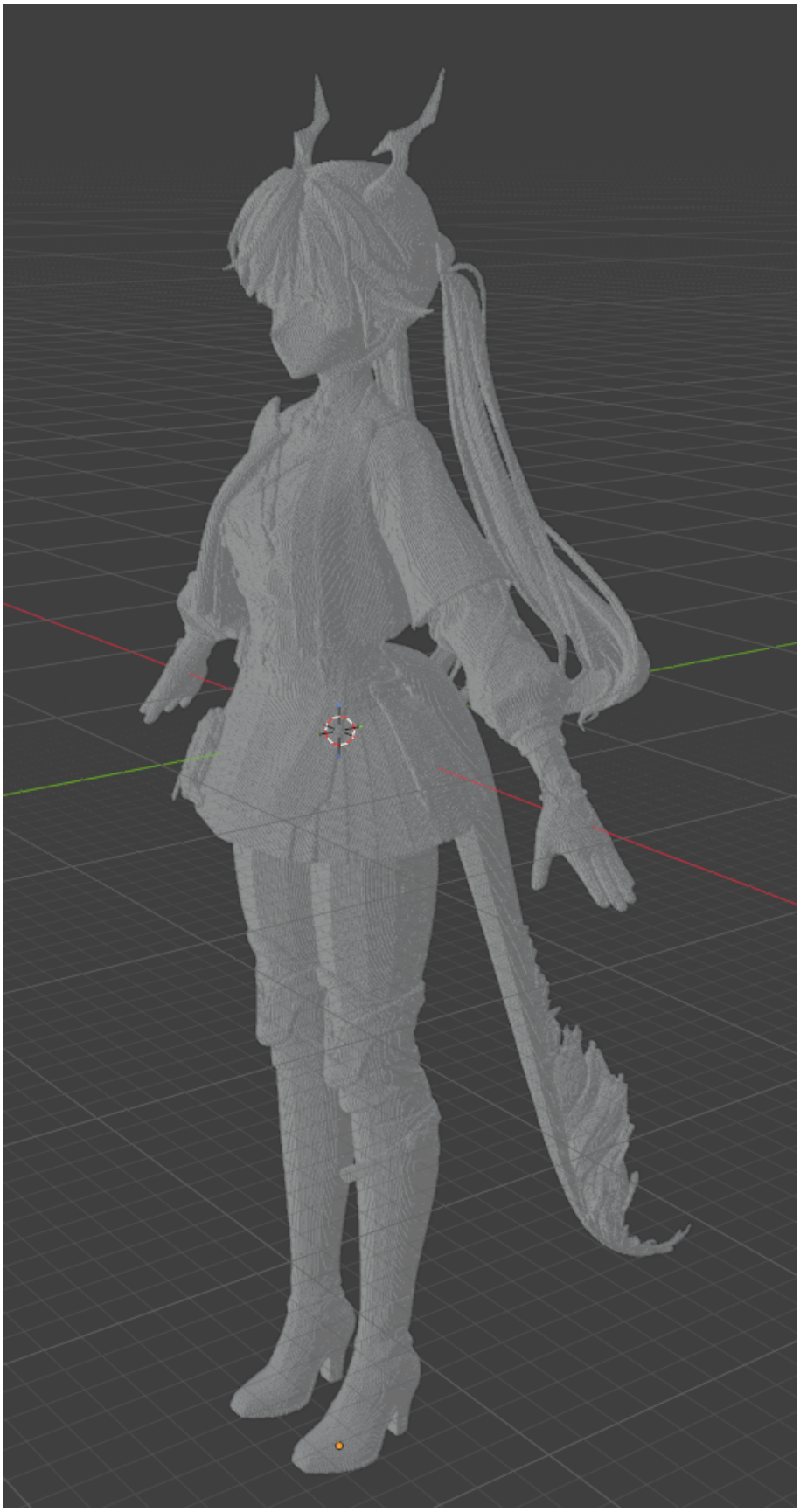
- Expected behavior

  Program menulis `output4.obj` dan mencetak jumlah *voxel*, *vertex*, dan *face*, statistik *node* per *depth*, jumlah *nodes not explored* per *depth*, serta *timings*.

- Hasil

```
$ .\build.bat run
Building ttov...
Enter file path: test/chen.obj

Subdivision level is the number of times the cube is subdivided into 8 smaller cubes.
For example, a subdivision level of 3 means the cube is subdivided into 8^3 = 512 smaller
Enter subdivision level: 10


============================================================
Scanning .obj file completed
Time taken to scan obj file: 131.023ms

Octree generation completed
Time taken to voxelize obj file: 2.3589979s

Octree to OBJ data type conversion completed
```

```
Time taken to convert octree to OBJ data type: 2.7613098s

OBJ data has been succesfully compacted
Time taken to compact OBJ data: 7.1535419s

Number of voxels: 1789487
Number of vertices: 5026538
Number of faces: 10736922

Number of nodes at each depth:
1       : 1
2       : 8
3       : 24
4       : 112
5       : 472
6       : 1856
7       : 8320
8       : 35240
9       : 158976
10      : 724192
11      : 3254280

Number of nodes not explored at each depth:
1       : 0
2       : 0
3       : 40
4       : 80
5       : 424
6       : 1920
7       : 6528
8       : 31320
9       : 122944
10      : 547616
11      : 2539256


OBJ file writing completed
Time taken to write obj file: 3.1406528s

Total time taken: 15.5455254s
```



Gambar 5: Test case 5

5.2 Ringkasan hasil pengujian

No.	File / scenario	Level L	Total time	Voxels	Notes
1	`test/cow.obj`	8	0,843 s	91 951	Stanford Cow
2	`test/pumpkin.obj`	9	13,98 s	1 196 633	
3	`test/teapot.obj`	8	0,916 s	118 263	
4	`test/line.obj`	5	0,013 s	288	
5	`test/chen.obj`	10	15,55 s	1 789 487	Chen Qianyu

Tabel 1: Ringkasan lima kasus uji

6 Kesimpulan

Program `ttov` yang ditulis dalam Go melakukan voxelization OBJ segitiga dengan *octree* dan *divide and conquer*, *pruning* cabang kosong, dan eksplorasi paralel di level atas. `output4.obj` berisi *surface voxels* dengan *uniform cube size* pada *max depth*, sesuai dengan spesifikasi I/O dan pelaporan statistik di CLI. Keterbatasan utama: ukuran file keluaran dan waktu komputasi membesar untuk *mesh* yang sangat padat atau L yang besar, serta asumsi OBJ minimal. Pengembangan lebih lanjut dapat berupa perapihan metrik *depth* di output teks atau heuristik untuk mengurangi uji segitiga per sel.

A Checklist penilaian

No.	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi OBJ terdiri dari kubus-kubus kecil	✓	
4	Program menampilkan informasi atau laporan sesuai spesifikasi	✓	
5	Program dapat menerima masukan OBJ dan menghasilkan keluaran	✓	
6	[Bonus] Program menggunakan konkurensi	✓	
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓

Catatan: *build* diuji pada Windows (`build.bat`) dan Linux (`Makefile`).